- *Message transformation*—To transform messages into specific types such as `SystemMessage`, `HumanMessage`, or `AIMessage`, use their respective classes. This ensures that messages are correctly formatted for processing within the chain.
- *Unique memory instances*—Be cautious not to share the same history or memory instance between different chains, as this can lead to confusion and inaccuracies in conversation history. Use a dedicated chain for handling the history to keep interactions straightforward and organized.

We now have a comprehensive toolkit to ingest, store, and manage information related to generative AI content. It would be helpful if we could use LLMs to effectively manage these tools, determining which ones to use and continuously invoking reasoning prompts. That's what agents can help us do.

6.4 Utilizing agents in LangChain.js

LangChain agents are abstractions that use LLMs as reasoning engines to determine and execute actions. Unlike standalone LLMs that simply generate text based on prompts, agents can interact with their environment, make decisions, and take actions based on those decisions. Think of agents as pieces of software that have a coded set of end goals and a list of tools that can be used to reach those goals. This capability makes them ideal for complex, multistep tasks that require reasoning and interaction with external tools or APIs.

For instance, a database-querying agent can respond to user inquiries like "What are the top three products in sales this month?" by querying a local database for sales data, analyzing the results to identify the highest-selling products, and returning the relevant details to the user. Similarly, a web search agent can handle questions such as "What are the latest advancements in AI?" by utilizing a search engine API to perform a query, retrieving and summarizing recent articles or papers on the topic, and presenting this information back to the user in a concise format.

Once you create an instance of an agent and provide all the necessary context and tools to achieve their goal, the agent software will follow a heuristic path to accomplish their goals while tracking each step of their progress for transparency.

We saw examples of tool calling using the Vercel AI SDK in chapter 4, but their simplicity is also their drawback. LangChain agents offer a more sophisticated abstraction. They build upon the concepts introduced by Vercel, such as tool composition and application context, but provide a more flexible and powerful framework for creating intelligent, action-oriented systems. LangChain agents improve upon the Vercel AI SDK's multistep design by offering a more dynamic decision-making process and providing built-in support for a wide range of tools and APIs out of the box. They also allow for more complex reasoning chains and iterative problem solving (by utilizing techniques like synergizing reasoning and acting in language models [ReAct, a prompting framework]).

Given the complexity of agents and intelligent workflow automation, our focus here will be on practical methods for creating agents with LangChain and integrating them into web applications.